



26. Principal Component Analysis

We can reduce the number of features in our data, but still feel like they are there?

Previous Section:

[🔗 25. Quadratic Discriminant Analysis](#)

Contents:

[1. Principal Component Analysis: The Overview](#)

[2. Principal Component Analysis is a Dimensionality Reduction Technique](#)

[3. Principal Component Analysis: The Comprehensive Workflow](#)

[4. Principal Component Analysis with Python](#)

[Summary](#)

[References](#)

This section is about **Principal Component Analysis**, a dimensionality reduction technique. But before even going further, let's take this very simple observation.

When you first look at data, it often feels heavy. Too many features. Too many dimensions. Too many directions to think in at once. Ten features? That's a 10-dimensional space. And let's be honest, we can't even *imagine* that properly.

So naturally, we ask: *"What if we just remove some features?"* And we *can* do that using classic techniques: Feature selection. Filtering. Dropping columns.

But something feels off. Because when you remove a feature. It's gone. Its information disappears with it. It's like un-equipping your armor in a game. You become lighter, but weaker.

Now here's where things get interesting. What if instead of removing features, we **transform them**? What if we take those 10 features, and compress them into 2. But somehow, the important information is still there.

Not exactly the same form. Not the same axes. But the same **essence**. It's like forging 10 pieces of gear into 2 legendary artifacts. Fewer items. But each one carries the power of many. That is the intuition behind PCA. It doesn't throw information away blindly. It **reorganizes** it.

It looks at the data silently, carefully. And asks a very different question:

┆ *"What are the most important directions in this space?"*

Not features. Not columns. **Directions**. Because sometimes, the way data is represented, is not the best way to understand it.

The original features? They might be redundant, correlated, overlapping in meaning. PCA doesn't accept that structure. It rotates the perspective. It finds new axes where the data reveals itself more clearly. Axes where:

- The first captures the most variation
- The second captures what remains
- And so on...

Until we decide: "This is enough". And suddenly, 10 dimensions become 2. Not by losing the data. But by **seeing it differently**.

1. Principal Component Analysis: The Overview

In Machine Learning, Principal Component Analysis is often introduced as a way to take a dataset with many features and turn it into something much smaller → Two dimensions. Three dimensions.

Something we can finally *see*. Something we can finally *understand*. You'll often see it like this:

- From 10 features to (X, Y)
- From 20 features to (X, Y, Z)

And suddenly, what was once impossible to visualize, becomes a simple plot. Clean. Interpretable. Almost intuitive.

But if you look closely, something deeper is happening here. This is not just about visualization. It's about **simplifying complexity without destroying it**.

Because normally, when we reduce features, we remove them. We drop columns. We discard information. And like I said before, that feels like un-equipping armor. You become lighter. But you lose something in return. PCA doesn't do that. It takes a completely different path. It keeps everything, but **rebuilds it**.

- Instead of asking: "Which features should we remove?". PCA asks: "How can we combine them better?"

And from that question, new features are born. Not original ones. Not copies. But something entirely new. These are called **Principal Components**

Each principal component is not a single feature. It is a **mixture**. A combination of all the original features, blended together in a very precise way.

Mathematically, they are linear combinations. But intuitively? They are **new perspectives** of the same data.

And here's the key idea. These new components are not random. They are carefully constructed so that:

- The first component captures the most important variation in the data
- The second captures what remains
- The third, even less
- And so on...

It's like compressing a story: The first sentence tells you the main idea → The second adds detail → The rest becomes optional.

There's another subtle property. One that makes PCA very powerful: Each principal component is **independent** from the others. No overlap. No redundancy. They don't repeat the same information. They don't "echo" each other like correlated features do. Instead, each one contributes something new.

So what PCA really does is this: It takes a messy, high-dimensional space. Full of overlapping, correlated features, and transforms it into a new space, where everything is clean, everything is uncorrelated, and most of the information is concentrated in just a few dimensions

That's why we can safely reduce dimensions. Not because the data is smaller. But because the **important information has been compressed**, squeezed into the first few components.

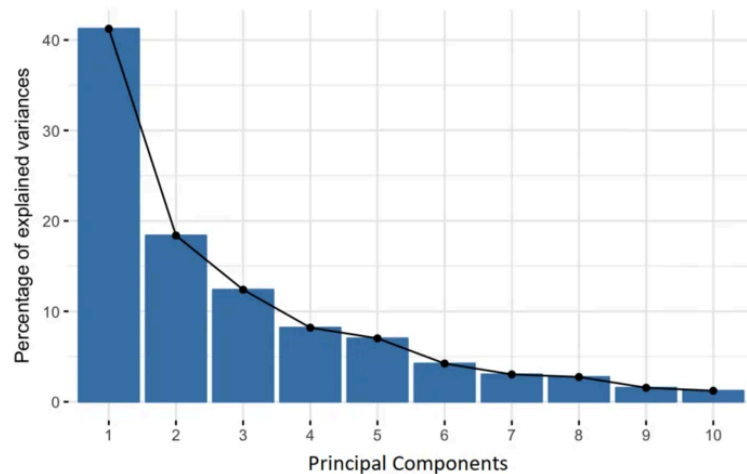
And this is why PCA becomes so useful for visualization (2D, 3D plots), for clustering (cleaner structure), and for simplifying models (less noise, less redundancy)

Even complex problems... Start to look... almost linear.

So in the end, PCA is not about removing features. It's about **redefining them**. Not losing information. But reorganizing it until the data finally reveals what actually matters.

(The following image depicts the percentages of explained variances for all the principal components, which is very important in selecting principal components in the PCA algorithm → We will revisit the explained variances

in the next section)



2. Principal Component Analysis is a Dimensionality Reduction Technique

(This part kinda extends from the previous section, so nothing really new is introduced here → Skip it if you want to)

What even is *dimensionality reduction*? At first, it sounds like a very technical term. But the idea behind it is actually simple. It's about this one problem:

“Too many features... too hard to understand.”

When your data has 10 features, you are working in a 10-dimensional space. 20 features? 20 dimensions. And at that point, you're no longer “seeing” the data. You're just trusting it exists.

So dimensionality reduction asks a very natural question

“Can we represent this data with fewer dimensions... without losing what actually matters?”

That's where Principal Component Analysis comes in.

Principal Component Analysis (PCA) is a dimensionality reduction technique widely used in machine learning and data preprocessing.

Its main objective is simple, but powerful:

- Take a dataset with many (often correlated) features
- Transform it into a smaller set of **uncorrelated variables**
- While preserving as much important information (variance) as possible

But here's something important, PCA does **not** just remove features. It doesn't throw anything away blindly. Instead, it **builds new features**. These are called **Principal Components**. And each one is a **linear combination** of the original features.

Now let's go back to intuition. Imagine you're playing an RPG game. You have 10 gears. Each one has its own stats. Its own name. Its own role. Now imagine this: You combine those 10 gears into just 2. And you name them PC1 and PC2 (Or maybe PC1, PC2, PC3 if you want 3 dimensions).

At first, that sounds impossible: “How can 2 items replace 10?”. But here's the trick: Each new gear carries the **combined stats** of the originals. Not individually, but collectively compressed.

That's exactly what PCA does. Given many features, it compresses them into a few principal components. But inside those components, the information of the original features is still there. Just reorganized.

Now, how does PCA decide what to keep? It follows a very strict rule:

■ **Keep the directions where the data varies the most**

So:

- The **first principal component (PC1)**: Captures the maximum variance and the most “important” direction in the data
- The **second principal component (PC2)**: Captures the next highest variance but must be **independent** (uncorrelated) from PC1
- The **third (PC3)** continues the pattern
- And so on...

Each new component adds less and less new information.

There's also a geometric way to see this: These components become new axes → PC1, PC2, PC3. → Just like X, Y, Z → But instead of being fixed, they are **learned from the data itself**.

And this is why PCA is so useful:

- For **visualization**: Reduce to 2D or 3D → plot and understand structure
- For **clustering**: Remove noise and redundancy → clearer groupings
- For **regression problems**: Reduce multicollinearity, simplify multiple regression into something more stable, fewer variables, but more meaningful ones

So in the end, Dimensionality reduction is not about shrinking data. It's about **making it understandable**. And PCA does this in a very elegant way:

- Not by removing features → But by **rebuilding them into something better**

Fewer dimensions. Same story. Just told more clearly.

3. Principal Component Analysis: The Comprehensive Workflow

Now we understand the idea of PCA. But how does it actually *work*? Not in theory, but step by step on real data. So let's slow down, and walk through it like how I often would:

Observe → Break it down → Reconstruct understanding

Consider the following student dataset:

Student	Height	Weight	Age	First_GPA	Second_GPA	Scholarship
P1	160	55	18	3.0	3.2	0
P2	161	56	18	3.1	3.3	0
P3	159	54	17	2.8	3.0	0
P4	162	57	18	3.2	3.4	0
P5	170	70	19	3.5	3.6	1
P6	169	72	19	3.4	3.5	1
P7	168	68	19	3.3	3.4	1
P8	172	75	20	3.6	3.7	1
P9	171	74	20	3.5	3.6	1
P10	167	69	19	3.2	3.4	0
P11	180	90	21	3.8	3.9	1
P12	178	85	21	3.7	3.8	1
P13	179	88	21	3.8	4.0	1
P14	181	92	22	3.9	4.0	1

Student	Height	Weight	Age	First_GPA	Second_GPA	Scholarship
P15	182	91	22	3.9	4.0	1
P16	190	95	23	4.0	4.0	1
P17	191	98	23	4.0	4.0	1
P18	150	45	17	2.2	2.5	0
P19	149	46	17	2.3	2.4	0
P20	151	47	17	2.4	2.6	0

Our goal is simple: Transform this 20×6 **dataset** → Into a 20×2 **dataset** (X, Y)

And one more important detail: The **Student column is not a feature**. It's just an identifier. So PCA ignores it completely (*but you might need to tell the model about this yourself*).

Step 1: Standardization

Before PCA even begins, we must make sure one thing is true. **All features must be on the same scale**. Because think about it:

- Height is around 150–190
- GPA is around 2.0–4.0
- Scholarship is just 0 or 1

If we don't fix this, the **Height** will dominate everything. Not because it's important. But because it's *large*.

So we standardize. We compute Mean and Standard deviation:

Feature	Mean	Std Dev
Height	169.5	12.42
Weight	71.3	17.58
Age	19.5	2.012
First_GPA	3.33	0.559
Second_GPA	3.46	0.511
Scholarship	0.6	0.502

(You can simply obtain these values using the Pandas library)

Then we apply the Z-score formula: $z = \frac{x - \mu}{\sigma}$

Example ($P1$ –Height): $z = \frac{160 - 169.5}{12.42} \approx -0.77$

And we repeat this for every feature. For every student. Until the dataset becomes a **standardized matrix**.

Z-Score Standardized Data

Student	Height	Weight	Age	First_GPA	Second_GPA	Scholarship
P1	-0.76	-0.68	-0.85	-0.60	0.04	-0.04
P2	-0.12	0.20	0.12	-0.20	0.85	0.68
P3	0.76	0.93	1.01	1.65	1.73	-1.57
P4	-1.65	-1.49	-0.93	-0.87	-0.99	-0.82
P5	-0.08	0.04	-0.19	0.21	0.15	-0.13
P6	1.06	0.78	0.95	1.17	1.12	1.35
P7	1.52	-1.50	-1.44	-1.39	-0.77	-0.77
P8	-1.27	-0.77	-0.27	-0.27	-0.27	0.22
P9	0.22	-0.27	0.72	0.72	0.72	1.22

Student	Height	Weight	Age	First_GPA	Second_GPA	Scholarship
P10	1.22	1.71	1.71	-1.27	-1.27	-1.27
P11	-0.59	-0.41	-0.95	-0.23	0.30	0.13
P12	-0.05	0.48	0.30	-0.23	0.84	0.66
P13	0.84	1.02	1.02	1.20	1.20	-2.02
P14	-1.84	-1.66	-0.52	-0.32	-0.91	-0.13
P15	0.26	0.07	-0.13	0.46	0.26	-0.13
P16	0.85	0.66	1.05	1.05	1.05	1.05
P17	1.05	-1.89	-2.08	-1.69	-1.19	-1.19
P18	-1.19	-1.19	0.80	0.80	0.80	0.80
P19	0.80	-1.19	0.80	0.80	0.80	0.80
P20	0.80	0.80	0.80	-1.19	-1.19	-1.19

At this point, the data is no longer biased by scale. Everything is fair. Everything is comparable. Now PCA can finally begin its real job.

Step 2: Compute the Covariance Matrix

Now we ask a deeper question: "How do these features move together?". Because PCA is not just about values. It's about **relationships**.

The covariance matrix captures exactly that.

- Positive values → move together
- Negative values → move in opposite directions
- Near zero → no relationship

Covariance Matrix (you can compute this using the `df.cov()` method in pandas)

Component	Height	Weight	Age	First_GPA	Second_GPA	Scholarship
Height	1.018	0.498	0.291	0.054	0.126	-0.221
Weight	0.498	1.093	0.787	0.357	0.364	-0.135
Age	0.291	0.787	0.980	0.573	0.448	0.136
First_GPA	0.054	0.357	0.573	0.944	0.819	0.327
Second_GPA	0.126	0.364	0.448	0.819	0.872	0.369
Scholarship	-0.221	-0.135	0.136	0.327	0.369	0.982

Look at it carefully. You'll start to notice patterns:

- Height and Weight → strongly related
- GPA features → highly correlated
- Scholarship → tied to performance

The data is not random. It has structure. And this is exactly what PCA is looking for. Not the raw numbers. But the **hidden relationships behind them**.

These first two steps may feel familiar: Standardization and Covariance. They are fundamental Machine Learning concepts.

But here, they are not just preparation. They are the **foundation**. Because everything PCA does next depends entirely on this structure.

Step 3: Find Eigenvalues and Eigenvectors

This is the part where many people pause. "Eigenvalues?" and "Eigenvectors?". Sounds intimidating. But don't worry. You don't need to master linear algebra here. You just need to understand **what role they play in PCA**.

At this stage, we already have something very important: The **covariance matrix**. Which means we already understand how features relate to each other.

Now PCA asks the next question: **"What are the most important directions hidden inside this matrix?"**. And this is exactly what eigenvalues and eigenvectors give us.

The Core Equation

$$\det(C - \lambda I) = 0$$

Where: C is the Covariance matrix; I is the Identity matrix; λ represents the eigenvalues → Solving this equation gives us the **eigenvalues**.

Eigenvalues (Importance of Each Direction)

Component	Eigenvalue
PC1	2.8268
PC2	1.6033
PC3	0.0490
PC4	0.2149
PC5	0.6354
PC6	0.5589

Think of eigenvalues like this. They measure **how much information (variance)** exists in a direction:

- Large eigenvalue → very important direction
- Small eigenvalue → less important

So even before doing anything else. We already know PC1 and PC2 matter the most

Eigenvectors (The Directions Themselves) → Now we solve:

$$(C - \lambda I)\mathbf{v} = 0$$

And we obtain eigenvectors. These are not just numbers. They are the **actual directions** in space.

Eigenvector Matrix

Feature	PC1	PC2	PC3	PC4	PC5	PC6
Height	-0.2424	-0.5255	-0.0491	0.1703	-0.7959	-0.0120
Weight	-0.4819	-0.3996	-0.2393	-0.6305	0.2865	0.2667
Age	-0.5105	-0.1089	0.3299	0.6279	0.3363	0.3334
First_GPA	-0.4746	0.3191	-0.6475	0.2309	0.0299	-0.4467
Second_GPA	-0.4488	0.3033	0.6312	-0.3375	-0.1686	-0.4072
Scholarship	-0.1470	0.5987	-0.1175	-0.1092	-0.3768	0.6725

Now here's the intuition. Each column (PC1, PC2, ...) is a **new axis**. And each value tells you: *"How much does this original feature contribute to that axis?"*

So at this point, PCA has done something incredible: It has discovered new directions. Built entirely from your original features.

Step 4: Feature Vectors (Selecting Principal Components)

Now comes a decision. We have **all components**. But if we use all of them, nothing changes → No dimensionality reduction happens

So we must choose: *"Which components do we keep?"*

The Rule is simple: Keep the components with the **largest eigenvalues**. Because they contain the most information.

We apply what is called the Explained Variance → Formula:

$$\text{Explained Variance} = \frac{\text{Eigenvalue}}{\text{Sum of all eigenvalues}}$$

Explained Variance Ratio

Component	Explained Variance
PC1	0.4801
PC2	0.2723
PC3	0.1079
PC4	0.0949
PC5	0.0365
PC6	0.0083

Now look carefully, PC1 alone is ~48% of the information; PC2 is ~27%. → Together, we have **~75% of the total variance**

So with just **2 components**, we already preserve most of the dataset. That's the power of PCA.

Selecting PC1 and PC2: We now construct the **feature vector matrix** by taking only the eigenvectors of PC1 and PC2.

Feature Vector Matrix (6 × 2)

Feature	PC1	PC2
Height	-0.2424	-0.5255
Weight	-0.4819	-0.3996
Age	-0.5105	-0.1089
First_GPA	-0.4746	0.3191
Second_GPA	-0.4488	0.3033
Scholarship	-0.1470	0.5987

This matrix is extremely important. Because this is what we will use to transform the data

Step 5: Transform the Data

Now everything comes together. We've: **Standardized the data** → **Understood relationships (covariance)** → **Found important directions (eigenvectors)** → **Selected the best ones (PC1, PC2)**

So what's left? Just one final step: ***Project the data into this new space***

The Transformation - We transform the data using:

$$\text{Final Data} = F^T \times Z^T$$

Where:

- F^T : Transpose of the feature vector matrix (selected eigenvectors)

- Z^T : Transpose of the standardized data

At first glance, this looks like just matrix multiplication. But conceptually? Something much more meaningful is happening.

We are taking each student, and asking: "Where does this student lie along the most important directions (PC1, PC2)?"

Dimensionality Change - If the original dataset is: $M \times N$ (20×6) → And we keep: $P = 2$ components. Then the new dataset becomes → $M \times P \rightarrow 20 \times 2$

From 6 features to just 2.

PCA-Transformed Dataset

Student	PC1	PC2
P1	1.2209	0.5627
P2	-0.5137	0.5716
P3	-2.4741	-0.7696
P4	2.5697	0.4967
P5	-0.0485	0.0783
P6	-2.3717	0.5483
P7	2.2073	-1.1773
P8	1.0374	0.9672
P9	-1.1346	1.0906
P10	-0.6402	-3.0588
P11	0.7806	0.6706
P12	-0.7391	0.3790
P13	-2.0246	-1.4241
P14	2.0927	1.2339
P15	-0.3501	-0.0016
P16	-2.1766	0.4549
P17	3.2340	-1.1854
P18	-0.3935	1.9895
P19	-0.8757	0.9439
P20	0.2950	-2.2800

What Does This Mean?

- Each student is no longer described by → Height, Weight, Age, GPA...
- Instead, they are described by just → **PC1 and PC2**

And here's the important part: These two new features are **uncorrelated, information-rich, and compact representations of the original data**

So even though we reduced: 6 dimensions → 2. We still preserved most of the structure. Most of the variance. Most of the story.

This is not just transformation. It's projection. Like shining a light on a 3D object, and capturing its shadow in 2D. You lose some detail. But the shape? The essence? Still there.

And now the data is finally ready. For visualization. For clustering. For learning.

Simpler. Cleaner. But still meaningful.

4. Principal Component Analysis with Python

PCA is straightforward in Python using the Scikit-Learn library

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

sample_data = {'Student': ['P1', 'P2', 'P3', 'P4', 'P5', 'P6',
                           'P7', 'P8', 'P9', 'P10', 'P11', 'P12',
                           'P13', 'P14', 'P15', 'P16', 'P17',
                           'P18', 'P19', 'P20'],
               'Height': [160, 161, 159, 162, 170, 169, 168, 172, 171, 167,
                          180, 178, 179, 181, 182, 190, 191, 150, 149, 151],
               'Weight': [55, 56, 54, 57, 70, 72, 68, 75, 74, 69,
                          90, 85, 88, 92, 91, 95, 98, 45, 46, 47],
               'Age': [18, 18, 17, 18, 19, 19, 19, 20, 20, 19,
                      21, 21, 21, 22, 22, 23, 23, 17, 17, 17],
               'First_GPA': [3.0, 3.1, 2.8, 3.2, 3.5, 3.4, 3.3, 3.6,
                             3.5, 3.2, 3.8, 3.7, 3.8, 3.9, 3.9,
                             4.0, 4.0, 2.2, 2.3, 2.4],
               'Second_GPA': [3.2, 3.3, 3.0, 3.4, 3.6, 3.5, 3.4, 3.7,
                              3.6, 3.4, 3.9, 3.8, 4.0, 4.0,
                              4.0, 4.0, 4.0, 2.5, 2.4, 2.6],
               'Scholarship': [0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
                               1, 1, 1, 0, 0, 0]}

# Get the columns and their values
df = pd.DataFrame(sample_data)
df = df.drop(columns=['Student'])
# Standardize the Data
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
```

```
# Calculate PCA with Scikit-Learn
from sklearn.decomposition import PCA

# Perform
pca = PCA(n_components=2) # Selecting top 2 principal components
principal_components = pca.fit_transform(df_scaled)
# Explained variance ratio
explained_variance_ratio = pca.explained_variance_ratio_
print("Explained Variance Ratio:\n", explained_variance_ratio)
# Principal Component Matrix
print("\nTransformed Data:\n", principal_components)
```

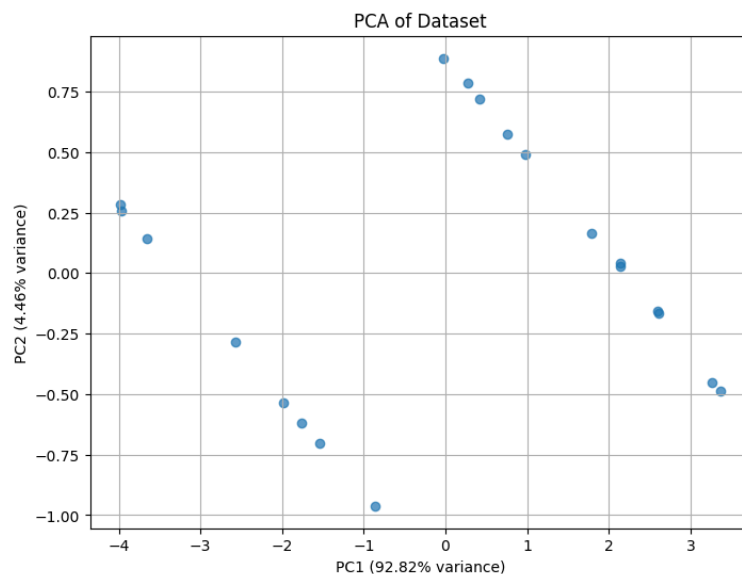
(You can run the code on any Python environment to see the output transformed data)

```
# Plot PCA
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.scatter(principal_components[:, 0], principal_components[:, 1], alpha=0.7)

plt.xlabel(f'PC1 ({explained_variance_ratio[0]:.2%} variance)')
plt.ylabel(f'PC2 ({explained_variance_ratio[1]:.2%} variance)')
```

```
plt.title('PCA of Dataset')
plt.grid(True)
plt.show()
```



Summary

Principal Component Analysis is sophisticated. But powerful. At first glance, it feels mathematical. Eigenvalues. Eigenvectors. Matrices.

But underneath all of that. The idea is surprisingly intuitive. PCA is not about removing data. It's about **understanding it better**. Not by looking at individual features. But by discovering the **directions that matter most**.

It takes a complex, high-dimensional dataset. Full of redundancy. Full of correlations. And transforms it into something: Smaller, cleaner; and easier to interpret.

- Instead of asking → "Which features should we drop?"
- PCA asks: "How can we represent this data more meaningfully?"

And the result is elegant:

New features (principal components) → Uncorrelated with each other → Ordered by importance (variance) → Capable of preserving most of the original information

So even when we reduce: 6 dimensions → 2. We are not losing the story. We are simply telling it in a clearer way.

And that's the beauty of PCA. Not complexity for the sake of complexity. But structure. Hidden inside the data. Waiting to be revealed.

References

<https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>

<https://www.ibm.com/think/topics/principal-component-analysis>

<https://builtin.com/data-science/step-step-explanation-principal-component-analysis>

<https://www.youtube.com/watch?v=FgakZw6K1QQ>

https://www.youtube.com/watch?v=_6UjscCJrYE

Next Section:

 [27. Independent Component Analysis](#)